

Discussions API

▼ Relevant source files

Backend/STUDER/src/main/java/facu/studer/controllers/UserController.java

Backend/STUDER/src/main/java/facu/studer/exceptions/GlobalExceptionHandler.java

Backend/STUDER/src/main/java/facu/studer/mappers/UserMapper.java

Backend/STUDER/src/main/java/facu/studer/repositories/UserRepository.java

The Discussions API facilitates a structured, threaded forum system within STUDER. It enables users to create topical discussions, participate via hierarchical message threads, manage favorites, and categorize content using a tag resolution system.

Architectural Overview

The discussion system is built on a layered architecture where `DiscussionController` exposes REST endpoints, delegating business logic to specialized services like `DiscussionService` and `DiscussionMessageService`.

Data Flow and Service Interaction

The following diagram illustrates the interaction between the API layer, the business logic, and the persistence layer for the discussion subsystem.

Discussion Subsystem Interaction

Sources: `facu/studer/controllers/DiscussionController.java` 1-25

`facu/studer/services/DiscussionService.java` 1-20

`facu/studer/services/DiscussionMessageService.java` 1-20

`facu/studer/services/TagService.java` 1-15

`facu/studer/repositories/DiscussionRepository.java` 1-15

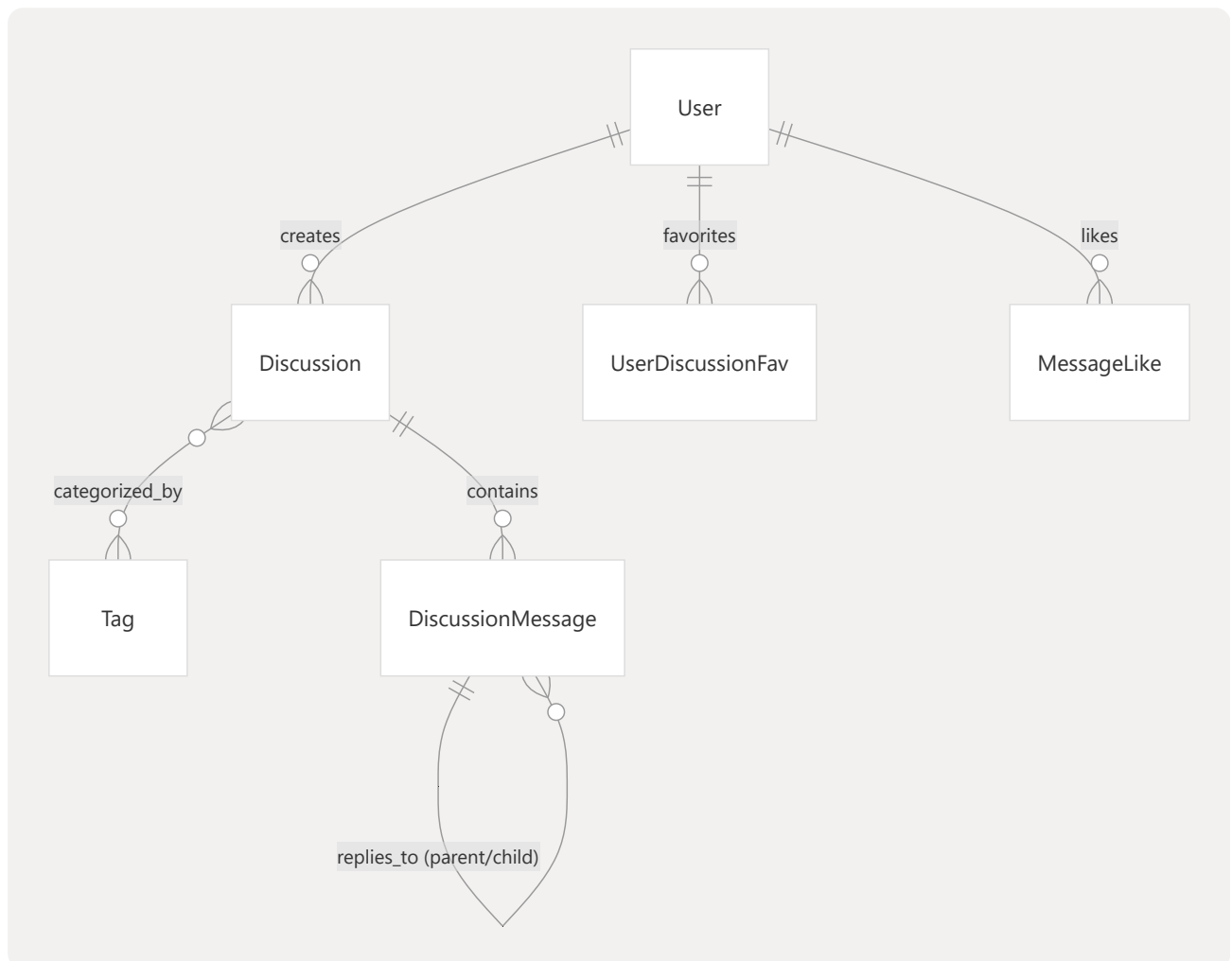
Discussion Entities & Tagging

Discussions are categorized by `Tag` entities. The system uses a many-to-many relationship between `Discussion` and `Tag`.

Entity Mapping

- **Discussion:** Contains metadata such as `title`, `description`, `isClosed` status, and the `User` who created it.
- **Tag:** A simple entity with a `name` used for filtering.
- **DiscussionMessage:** Represents a single post within a discussion, supporting a `parentMessage` reference for threading.

Entity Relationship Diagram



Sources: `facu/studer/entities/Discussion.java`

`facu/studer/entities/DiscussionMessage.java`

`facu/studer/entities/Tag.java`

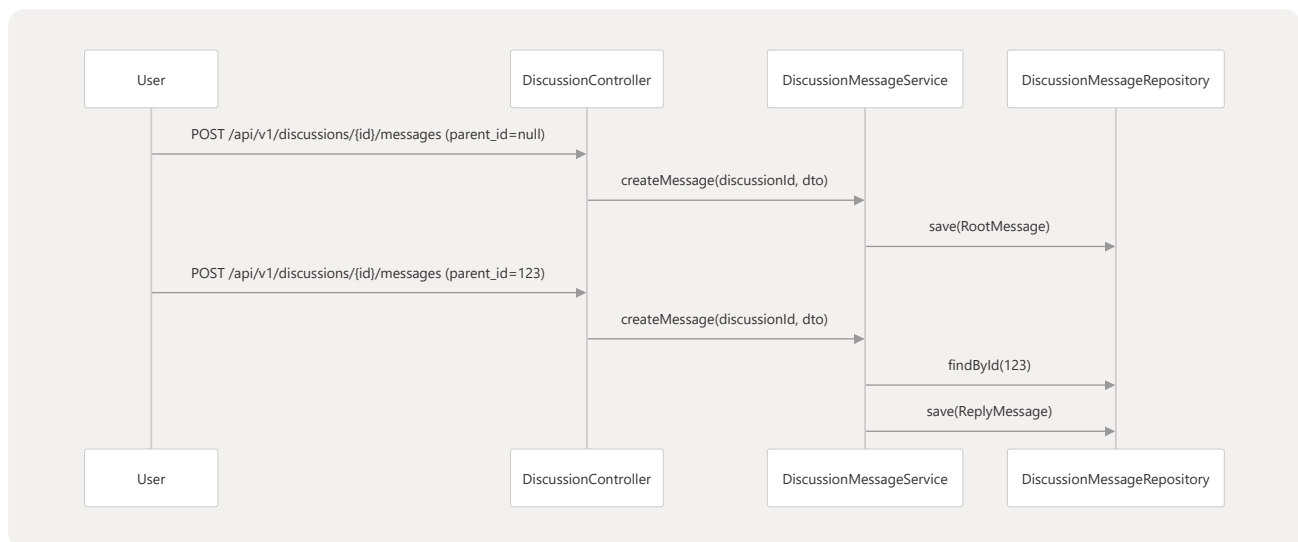
Threaded Message System

The `DiscussionMessageService` manages the lifecycle of messages. It supports a hierarchical structure where each message can have a `parent_id`, allowing for nested replies.

Implementation Details

- **Hierarchy:** Messages are stored in a single table with a self-referencing foreign key `parent_id`.
- **Ordering:** Messages are typically ordered by creation date or "likes" count via the `MessageLikeService`.
- **State Management:** If a `Discussion` is marked as `isClosed = true`, the `DiscussionMessageService` prevents the creation of new messages.

Message Hierarchy Flow



Sources: `facu/studer/services/DiscussionMessageService.java` 1-40

`facu/studer/controllers/DiscussionController.java` 50-70

Tag Resolution (TagService)

When a discussion is created or updated, the `TagService` is responsible for resolving the list of tag names provided in the request into `Tag` entities.

- **Logic:** It checks `TagRepository` for existing tags by name.
- **Persistence:** If a tag does not exist, it is created on-the-fly to ensure the many-to-many relationship is maintained.

Sources: `facu/studer/services/TagService.java` 1-25

`facu/studer/repositories/TagRepository.java` 1-15

Favorites and Likes

User engagement is tracked through two distinct services:

1. **UserDiscussionFavService:** Manages the `UserDiscussionFav` entity, allowing users to bookmark discussions.
2. **MessageLikeService:** Manages `MessageLike` entities. It ensures idempotency (a user can only like a message once) and provides counts to the `DiscussionMessageService` for display.

Sources: `facu/studer/services/UserDiscussionFavService.java` 1-20

`facu/studer/services/MessageLikeService.java` 1-20

API Endpoints Summary

The `DiscussionController` exposes the following primary endpoints:

Method	Path	Description
GET	<code>/api/v1/discussions</code>	List discussions (supports pagination and tag filters).
POST	<code>/api/v1/discussions</code>	Create a new discussion with tags.
GET	<code>/api/v1/discussions/{id}</code>	Get discussion details and top-level messages.
PUT	<code>/api/v1/discussions/{id}/close</code>	Mark a discussion as closed (Admin or Creator).
POST	<code>/api/v1/discussions/{id}/messages</code>	Post a message or reply to a thread.
POST	<code>/api/v1/discussions/{id}/fav</code>	Toggle favorite status for the authenticated user.

Sources: facu/studer/controllers/DiscussionController.java 30-120

Error Handling

Discussions leverage the `GlobalExceptionHandler` to manage domain-specific errors:

- **DiscussionClosedException:** Thrown when attempting to post a message to a closed discussion.
- **ResourceNotFoundException:** Thrown when a requested `DiscussionId` or `Tag` does not exist.
- **Validation Error:** Handled via `MethodArgumentNotValidException` for invalid DTO fields

facu/studer/exceptions/GlobalExceptionHandler.java 31-53

Sources: facu/studer/exceptions/GlobalExceptionHandler.java 1-91